

MFP — Questions théoriques

Tests, couverture de code et approche TDD

22/03/2026

1. Codewars et l'approche TDD

Comment les tests sont-ils utilisés dans la résolution d'algos Codewars ?

Sur Codewars, lorsqu'on reçoit un kata (un exercice algorithmique), les tests sont déjà écrits avant même que l'on commence à coder. On voit immédiatement les cas d'usage attendus : quels inputs produisent quels outputs, quels cas limites sont testés (chaîne vide, tableau vide, valeurs négatives, etc.).

C'est exactement le principe du TDD (Test Driven Development) appliqué à la résolution d'algorithmes :

1. **Red** — les tests échouent car la fonction n'est pas encore implémentée
2. **Green** — on écrit le minimum de code pour faire passer les tests
3. **Refactor** — on optimise la solution sans casser les tests

Les tests Codewars servent donc à plusieurs choses simultanément : ils définissent le contrat de la fonction (ce qu'elle doit faire), ils guident l'implémentation (on sait exactement quand on a fini), et ils protègent contre les régressions si on refactorise.

La différence principale avec un vrai projet TDD est que sur Codewars les tests sont écrits par quelqu'un d'autre, alors qu'en TDD on les écrit soi-même avant de coder. Mais la philosophie est identique : les tests précèdent et guident le code.

2. Test d'intégration vs test unitaire

Quelles sont les différences entre tester le controller avec un test d'intégration, ou tester la fonction de recherche avec des tests unitaires ? Quelle conclusion en tirer ?

Test unitaire — countOccurrences

Un test unitaire cible une fonction isolée, sans aucune dépendance externe. Dans notre projet, tester countOccurrences ressemble à ceci :

```
expect(countOccurrences("hello world hello", "hello")).toBe(2)
```

Avantages du test unitaire :

- Extrêmement rapide à exécuter (millisecondes)
- Pas de base de données, pas de réseau, pas de serveur à démarrer
- Si le test échoue, on sait exactement où est le bug — dans la fonction elle-même
- Facile à écrire et à maintenir
- Permet de tester tous les cas limites (chaîne vide, mot inexistant, casse différente)

Limite du test unitaire :

- Ne teste pas que la fonction est correctement branchée dans le reste de l'application
- Une fonction peut passer tous ses tests unitaires mais être mal appelée dans le controller

Test d'intégration — POST /search

Un test d'intégration teste le système de bout en bout via HTTP. On envoie une vraie requête au serveur et on vérifie la réponse :

```
await request(app).post("/search").send({ text: "hello", word: "hello" })
```

Avantages du test d'intégration :

- Teste tout le pipeline : routing, parsing du body, logique, format de la réponse
- Vérifie que les pièces s'assemblent correctement
- Plus proche de ce qu'un vrai client (frontend, Bruno) ferait
- Détecte des bugs de configuration que les tests unitaires ne verraient pas

Limites du test d'intégration :

- Plus lent à exécuter
- Si le test échoue, il faut investiguer pour savoir si c'est le routing, le controller ou la logique
- Plus difficile à mettre en place (base de données, serveur, mocks)

Conclusion

Les deux approches sont complémentaires et nécessaires. La règle générale est celle de la pyramide de tests : une majorité de tests unitaires (rapides, précis, nombreux), moins de tests d'intégration (plus lents mais plus réalistes), et encore moins de tests E2E.

Pour une fonction pure comme `countOccurrences`, le test unitaire est suffisant et préférable. Pour valider que l'endpoint `/search` répond correctement avec le bon status code et le bon format JSON, le test d'intégration est indispensable. Les deux ensemble donnent une couverture solide et équilibrée.

3. Pourquoi 100% de coverage ne signifie pas une application totalement testée

Pourquoi dans ce projet avoir un code coverage de 100% ne signifie pas avoir une application totalement testée ?

Le code coverage mesure quelles lignes de code ont été exécutées pendant les tests. Atteindre 100% signifie que chaque ligne a été traversée au moins une fois — mais cela ne dit rien sur la qualité ou l'exhaustivité des vérifications effectuées.

1. Le coverage ne mesure pas la qualité des assertions

Une ligne peut être exécutée sans qu'on vérifie vraiment son résultat. Par exemple :

```
test('passe', () => { getDistance(Paris, Lyon); })
```

Cette ligne couvre `getDistance` à 100% mais ne vérifie pas que le résultat est correct. Le coverage est à 100%, la logique n'est pas testée.

2. Les tests E2E Playwright ne sont pas comptabilisés

Dans notre projet, les tests Playwright testent l'interface utilisateur dans un vrai navigateur. Ces tests exercent les endpoints de l'API, mais Jest ne voit pas ces appels — ils ne contribuent pas au coverage. Un endpoint pourrait être cassé côté UI sans que le coverage baisse.

3. `index.ts` est à 0% et c'est normal

Le fichier `index.ts` qui démarre le serveur et initialise la base de données est impossible à tester unitairement. Il restera toujours à 0% de coverage. Atteindre 100% global impliquerait soit de le tester de façon artificielle, soit de l'exclure — ce qui fausse la métrique.

4. Les cas métier complexes ne sont pas couverts

Le coverage ne teste pas les scénarios réels d'utilisation. Par exemple :

- Que se passe-t-il si deux utilisateurs créent simultanément un compte avec le même email ?
- Que se passe-t-il si la base de données est pleine ?
- Que se passe-t-il si `getCoordinatesFromSearch` retourne des coordonnées invalides ?
- Que se passe-t-il si le token JWT expire pendant une session ?

Ces scénarios peuvent représenter des bugs critiques en production sans jamais apparaître dans le coverage.

5. Les tests peuvent être trop couplés à l'implémentation

Si on écrit des tests qui reproduisent exactement la logique interne plutôt que de tester le comportement attendu, on peut avoir 100% de coverage avec des tests qui passent même quand le code est faux. C'est le cas si on mock trop agressivement ou si on teste des détails d'implémentation plutôt que des comportements.

Conclusion

Le coverage est un indicateur utile pour détecter les zones non testées, mais c'est un plancher et non un plafond. Un coverage de 80% avec des assertions solides sur les cas critiques vaut mieux qu'un coverage de 100% avec des tests superficiels. Dans notre projet MFP, les vraies garanties de qualité viennent de la combinaison des tests unitaires sur les fonctions pures, des tests d'intégration sur les endpoints, et des tests E2E Playwright sur les parcours utilisateur complets.

4. Pipeline CI — GitHub Actions

Le pipeline CI exécute automatiquement les tests à chaque push sur les branches dev et master.

Oui, le workflow GitHub Actions mis en place est un pipeline CI (Continuous Integration). Son rôle est de garantir qu'aucun code cassé n'est intégré dans les branches principales sans que les tests aient été vérifiés.

Configuration du pipeline

Le fichier `.github/workflows/tests.yml` définit le pipeline. Il se déclenche sur chaque push vers dev ou master et exécute les étapes suivantes :

- Checkout du code source depuis GitHub
- Installation de Node.js version 20
- Installation des dépendances avec yarn install dans le dossier server/
- Exécution de yarn test qui lance Jest avec la couverture de code

Si un seul test échoue, le pipeline marque le run comme KO. On peut voir le résultat dans l'onglet Actions du dépôt GitHub.

Valeur ajoutée du CI

Sans CI, un développeur pourrait pousser du code qui casse des tests existants sans s'en rendre compte. Avec le pipeline, chaque push est automatiquement validé dans un environnement propre et reproductible (Ubuntu, Node 20).

5. Fonctionnalité développée en TDD : notation et visibilité des adresses

Développez une nouvelle fonctionnalité dans une approche TDD avec des commits qui montrent bien la temporalité.

La fonctionnalité développée en TDD est la notation et la gestion de la visibilité des adresses. Un utilisateur peut attribuer une note de 1 à 5 à une adresse sauvegardée, et choisir de la rendre publique ou privée.

Étape 1 — Red : écriture des tests avant le code

Les tests ont été écrits en premier, avant toute implémentation. Deux types de tests ont été créés :

- Tests unitaires dans `tests/validateRating.test.ts` et `tests/validateVisibility.test.ts`
- Tests d'intégration dans `src/app.spec.ts` pour les endpoints `PATCH /api/addresses/:id/rating` et `PATCH /api/addresses/:id/visibility`

Commit : `test: add failing tests for address rating and visibility features (TDD step 1)` — à ce stade `yarn test` retourne des erreurs, c'est le comportement attendu.

Étape 2 — Green : implémentation minimale

- `validateRating` : vérifie que la valeur est un entier entre 1 et 5
- `validateVisibility` : vérifie que la valeur est un booléen
- `PATCH /api/addresses/:id/rating` : met à jour la note d'une adresse
- `PATCH /api/addresses/:id/visibility` : bascule la visibilité publique/privée
- `GET /api/addresses/public` : retourne toutes les adresses publiques

Commits : `feat: implement validateRating and validateVisibility utilities (TDD step 2)` et `feat: implement rating and visibility endpoints on addresses (TDD step 2)`

Étape 3 — Refactor : amélioration de la couverture

Des tests supplémentaires ont été ajoutés pour couvrir les cas limites : note invalide, adresse inexistante, tentative de modification d'une adresse appartenant à un autre utilisateur (403).

Commit : `test: add coverage tests for missing branches (TDD step 3 - refactor)`

Correction de bugs en TDD : validation de l'email

Un bug a été identifié : il était possible de créer un compte avec un email invalide ou déjà utilisé. La correction a suivi la même approche TDD :

- Commit 1 — tests échouants pour les deux bugs
- Commit 2 — implémentation de `validateEmail`
- Commit 3 — correction du controller `Users.ts` avec les deux validations
- Commit 4 — fichier dédié `tests/emailValidation.test.ts`

6. Tests Bruno pour un collègue : suppression de compte

Réfléchissez à une fonctionnalité qui devra être développée par un collègue, écrivez les tests Bruno sans développer la fonctionnalité.

La fonctionnalité choisie est la suppression de compte (DELETE /api/users/me). Elle n'est pas encore implémentée.

Tests Bruno écrits

- delete-account.bru — vérifie que DELETE /api/users/me retourne 200 avec un message de confirmation
- delete-account-unauthorized.bru — vérifie que l'endpoint retourne 403 sans token
- delete-account-verify.bru — vérifie que le token est invalidé après suppression
- delete-account-addresses-cleaned.bru — vérifie que les adresses sont également supprimées

Ce que ces tests définissent pour le collègue

- Créer la route DELETE /api/users/me dans usersRouter
- Supprimer toutes les adresses de l'utilisateur avant de supprimer le compte
- Retourner un status 200 avec un champ message
- La route doit être protégée par isAuthenticated

C'est l'utilité des tests comme contrat : ils remplacent une spécification écrite en prose par des assertions exécutables et vérifiables.